Project Report On

LIPI - Script Learning Application

Submitted in partial fulfillment of the requirements for the award of the degree of

Bachelor of Technology

in

Computer Science and Engineering

By

Abhinav Nair A. (U2203009)

Anantharam S. (U2203040)

Athul K. (U2203060)

Bevan Sony Joseph (U2203067)

Under the guidance of

Ms. Sherine Sebastian

Department of Computer Science and Engineering
Rajagiri School of Engineering & Technology (Autonomous)
(Parent University: APJ Abdul Kalam Technological University)
Rajagiri Valley, Kakkanad, Kochi, 682039

March 2026

CERTIFICATE

This is to certify that the project report entitled ***"LIPI - Script Learning Application"*** *is a bonafide record of the work done by* ***Abhinav Nair A. (U2203009), Anantharam S. (U2203040), Athul K. (U2203060)*** *and* ***Bevan Sony Joseph (U2203067)****, submitted to the Rajagiri School of Engineering & Technology (RSET) (Autonomous) in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology (B. Tech.) in "Computer Science and Engineering" during the academic year 2025-2026.*

Project Guide
Ms. Sherine Sebastian
Assistant Professor
Dept. of CSE
RSET

Project Co-ordinator
Dr. Jisha G.
Associate Professor
Dept. of CSE
RSET

Dr. Preetha K.G.
Head of the Department
Dept. of CSE
RSET

ACKNOWLEDGMENT

We wish to express our sincere gratitude towards **Rev. Dr. Fr. Jaison Paul Mulerikkal CMI**, Principal of RSET, and **Dr. Preetha K.G.**, Head of the Department of Department of Computer Science and Engineering for providing us with the opportunity to undertake our project, "Lipi - Script Learning Application".

We are highly indebted to our project coordinators, **Dr. Jisha G.**, Associate Professor, Department of Computer Science and Engineering, and **Dr. Preetha K.G.**, Professor, Department of Computer Science and Engineering, for their valuable support.

It is indeed our pleasure and a moment of satisfaction for us to express our sincere gratitude to our project guide **Ms. Sherine Sebastian**, Assistant Professor, Department of Computer Science and Engineering, for her patience and all the priceless advice and wisdom she has shared with us.

Last but not the least, we would like to express our sincere gratitude towards all other teachers and friends for their continuous support and constructive ideas.

Abhinav Nair A.

Anantharam S.

Athul K.

Bevan Sony Joseph

Abstract

The project Lipi – Script Learning Application is an interactive and structured mobile application designed to help users learn to read and write Indian regional scripts, starting with Malayalam. The app addresses the lack of structured and engaging tools for script learning by integrating guided tracing, audio pronunciation, and machine learning–based handwriting evaluation for regional Indian languages like Malayalam.

Learners are introduced to each alphabet through visual stroke animations and native pronunciations, followed by touch-based tracing practice through a predefined fixed path. Using a custom Convolutional Neural Network (CNN) model, with a modular approach, the app provides real-time feedback by analyzing the accuracy of user-drawn letters. A progress dashboard, gamified rewards, and an overall relaxed UI, enhance engagement and motivate continuous learning. Built using Flutter for the frontend and Python for the ML backend, Lipi – Script Learning Application offers a scalable and accessible platform. By combining visual, auditory, and ML-driven feedback system, this project promotes digital literacy, cultural preservation, and inclusive education for learners of all ages.

Contents

Acknowledgment **i**

Abstract **ii**

List of Abbreviations **vi**

List of Figures **vii**

List of Tables **viii**

1 Introduction **1**

1.1 Background 1

1.2 Problem Definition 2

1.3 Scope and Motivation 2

1.4 Objectives 3

1.5 Challenges 3

1.6 Assumptions 4

1.7 Societal / Industrial Relevance 4

1.7.1 Societal Relevance 4

1.7.2 Industrial Relevance 5

1.8 Organization of the Report 5

1.9 Summary of Chapter 6

2 Literature Survey **7**

2.1 Chauhan, R., Sharma, P. (2023). HCR-Net: A deep learning based script-independent handwritten character recognition network. IEEE Transactions on Pattern Analysis and Machine Intelligence.[1] 7

2.2 Sinha, A., Gupta, V. (2023). Enhancing handwritten Devanagari character recognition via GAN-generated synthetic data. Journal of Artificial Intelligence Research.[2] 8
2.3 Reddy, K., Iyer, S. (2022). Nearest quad-tree (NQT) classifier: A novel framework for handwritten character recognition. International Journal of Computer Vision and Image Processing.[3] 9
2.4 Lakshmi, R., Kumar, M. (2023). A Lightweight CNN Model for Handwritten Character Recognition for Tamil Language. Procedia Computer Science, Elsevier.[4] 10
2.5 Comparison of Reviewed Literature 11
2.6 Research Gaps Identified 12
2.7 Summary of Chapter 12

3 System Design **13**
3.1 Overall Architecture 13
3.2 Design Diagrams 14
3.2.1 Architecture Diagram 14
3.2.2 System Workflow 15
3.2.3 Sequence Diagram 16
3.3 Module Division 16
3.3.1 Module 1 - Script Visualization and Guidance 16
3.3.2 Module 2 - Interaction and Tracing Module 17
3.3.3 Module 3 - Machine Learning Evaluation Module 17
3.3.4 Module 4 - Feedback and Progress Tracking Module 17
3.4 Tools and Technologies 17
3.4.1 Tracing Tool 17
3.4.2 Machine Learning Model 18
3.4.3 Dataset 19
3.4.4 Audio and Pronunciation Support 20
3.5 Work breakdown and Responsibilities 21
3.6 Key Deliverables 21
3.7 Project Timeline 22

3.8 Summary of Chapter 23

4 Results and Discussions **24**

4.1 Implementation Results 24

4.2 Model Performance Analysis 25

4.3 Model Performance Analysis 28

4.4 System Performance Evaluation 29

4.5 Discussion 29

5 Conclusions & Future Scope **30**

5.1 Conclusion 30

5.2 Future Scope 30

References **31**

List of Publications **33**

Appendix A: Presentation **34**

Appendix B: Vision, Mission, Programme Outcomes and Course Outcomes **61**

Appendix C: CO-PO-PSO Mapping **65**

List of Abbreviations

CNN - Convolutional Neural Network

SVG - Scalable Vector Graphics

UI - User Interface

ML - Machine Learning

HCR - Handwritten Character Recognition

AI - Artificial Intelligence

IPA - International Phonetic Alphabets

Ed-Tech - Educational Technology

List of Figures

3.1 Overall System Architecture illustrating the interaction between the Mobile App (Flutter), Backend Services with its ML Engine, and the Data Layer. 14
3.2 Sequence Diagram showing the step-by-step interactions for a letter tracing session, ML model evaluation, and progress management. 16
3.3 Architecture of the CNN proposed in the project 19
3.4 Overall System Architecture illustrating the interaction between the Mobile App (Flutter), Backend Services with its ML Engine, and the Data Layer. 20

4.1 Training and Validation Accuracy Across Epochs 25
4.2 Training and Validation Loss Across Epochs 26
4.3 Confusion Matrix for Malayalam Character Classification 27

List of Tables

2.1 Comparison of Reviewed Literature on Handwritten Character Recognition 11

Chapter 1

Introduction

In a country like India where there exists over 19,000 mother tongues, learning to read and write regional scripts presents a significant challenge, particularly for children and non-native speakers. Many traditional and digital learning platforms focus mainly on reading and pronunciation, offering limited guidance on the actual writing process. This gap often makes script learning difficult, less engaging, and inconsistent.

The project "Lipi – Script Learning Application" aims to bridge this gap by providing an interactive, guided and an enjoyable way to learn regional Indian scripts, starting with Malayalam. The app enables users to trace each letter through step-by-step stroke animations, listen to native pronunciation, and practice freehand writing, with feedback, within the same platform.

A key addition to Lipi is its use of machine learning to evaluate the accuracy of handwriting. After the user traced or wrote a character, a lightweight Convolutional Neural Network (CNN) model analyzes the stroke and provides real-time feedback on accuracy and correctness. This ensures that users not only memorize characters, but also master their proper formation, which helps them write that letter with confidence.

The app is designed with accessibility and engagement in mind, featuring a fun interface with progress tracking and visual–auditory learning aids. By combining technology and pedagogy, Lipi promotes script literacy, preserves cultural identity, and enhances the learning experience for users of all ages.

1.1 Background

India is home to a rich variety of languages and scripts, each representing unique cultural and linguistic traditions. Despite this diversity, digital tools for teaching Indian scripts remain limited, particularly those that focus on writing practice and stroke accuracy.

Traditional learning methods often rely on static visuals, which do not effectively teach learners the complex stroke patterns or curves that are usually seen in Indian languages. With the growth of mobile technology and digital education, there is an increasing need among children and even adults for learning applications that make learning languages engaging and accessible. Existing language learning platforms provide exposure to pronunciation and vocabulary, but lack features for script writing practice, guided tracing, and real-time evaluation. Moreover, most script-learning apps focus only on recognition rather than generation, leaving a gap in tools that support the entire learning cycle—from visual guidance to verification.

Lipi addresses this need by combining interactive UI design, guided tracing, and machine learning–based handwriting test into a single educational platform. Through this integration, the app not only improves script literacy but also contributes to the preservation of regional languages by making the process of learning them more immersive and enjoyable.

1.2 Problem Definition

There is a lack of effective digital platforms that teach Indian scripts through guided writing practice and feedback. Learners face difficulty understanding stroke order and accuracy, highlighting the need for an interactive, ML - based assisted tool that makes script learning structured, engaging, and accessible.

1.3 Scope and Motivation

The scope of "Lipi – Script Learning Application" extends to creating a complete and interactive platform for learning Indian scripts, starting with Malayalam. The app integrates guided tracing, pronunciation support, and AI-based handwriting evaluation to deliver a complete learning experience. It is designed for children, beginners, and non-native learners who wish to understand and practice script writing systematically. By combining visual, auditory, and a feedback-based learning methods, the app aims to make script literacy more accessible and enjoyable for diverse user groups.

The motivation behind developing Lipi arises from the lack of engaging and accurate tools that teach users how to write Indian alphabets correctly. Traditional methods often fail to provide real-time feedback or maintain learner interest, especially among younger

audiences. With advancements in mobile technology and AI, there is an opportunity to create an educational tool that adapts to each learner's pace and progress. The project seeks to preserve linguistic heritage by encouraging people to learn and write regional scripts in a modern, digital environment. Ultimately, Lipi aspires to make script learning both effective and culturally meaningful.

1.4 Objectives

- To develop an interactive mobile application that teaches users to write Indian regional scripts, beginning with Malayalam.
- To implement guided stroke animations that visually demonstrate the correct order and direction for each alphabet.
- To integrate a touch-based tracing interface that allows users to practice writing letters using finger or stylus input.
- To design and train a lightweight custom CNN model, with a modular approach, for real-time handwriting accuracy evaluation.
- To incorporate audio pronunciation for each alphabet to support auditory learning and improve pronunciation skills.
- To create a progress tracking system with visual feedback, badges, and scores that motivate continued learning and engagement.

1.5 Challenges

Developing "Lipi" involves challenges such as handling variations in individual handwriting styles, which can affect the accuracy of the CNN-based evaluation model. Ensuring smooth real-time performance on low-end mobile devices while maintaining responsive animations and instant feedback is another major concern. Additionally, creating a diverse and high-quality dataset for training the model poses difficulties in achieving reliable recognition for different users.

1.6 Assumptions

- **Technology Accessibility**: It is assumed that the target users, including children and beginners in diverse communities, have access to smartphones running at least Android 6.0 or higher, which is the minimum supported version for the application.
- **Dataset Feasibility**: The project assumes that a high-quality, diverse dataset for training the machine learning model can be successfully created.
- **On-Device Performance**: The design assumes that the custom Convolutional Neural Network (CNN) model can be optimized to run efficiently on mobile devices with limited resources.
- **Teaching Effectiveness**: The project is founded on the assumption that the proposed learning method—combining guided tracing animations, ML-based feedback, and audio pronunciation—is an effective and engaging way for users to learn complex scripts like Malayalam.

1.7 Societal / Industrial Relevance

1.7.1 Societal Relevance

- **Cultural Preservation:** By creating an accessible tool to learn Indian regional scripts like Malayalam, the app plays a vital role in promoting literacy and preserving cultural identity. It ensures that complex, traditional writing systems remain learnable for new generations.
- **Enhanced Educational Equity:** The application is designed for learners of all ages, especially children, beginners, and non-native speakers. This focus on accessibility helps bridge learning gaps and supports educational equity in communities where such specialized tools are scarce.
- **Improved Learning Accessibility:** The project addresses a clear need for a digital solution that provides structured and ordered instruction for writing alphabets, a feature that current online platforms often lack. The intuitive UI is specifically designed for children and beginners, making learning more accessible.

1.7.2 Industrial Relevance

- **Growth in the Ed-Tech Sector:** The app is a prime example of an innovative product for the booming Ed-Tech market. Its focus on interactive, AI-driven language learning aligns with the industry's shift towards personalized and effective digital educational experiences.

- **Scalable and Modern Technology Stack:** The use of Flutter for the front-end and a Python ML back-end represents a modern, cost-effective, and scalable architecture. This modular design creates a scalable product with the potential to tap into a much larger market.

- **Data-Driven Product Improvement:** The system's architecture includes an analytics component for tracking user progress and usage. This data is invaluable for iterative product development, allowing for data-driven improvements to the learning model and user experience, which is a standard and vital practice in the software industry.

1.8 Organization of the Report

This report is structured as follows:

Chapter 1: Introduction This chapter provides a comprehensive overview of the Script Learning App project. It outlines the problem statement, project objectives, and the societal and industrial relevance of developing an interactive, ML-driven application for learning Indian regional scripts.

Chapter 2: Literature Survey This chapter reviews existing research and technologies relevant to handwritten character recognition, mobile learning applications, and user interface design for educational tools.

Chapter 3: System Design This chapter details the technical architecture and design of the application.

Chapter 4: Conclusion This chapter summarizes the key aspects of the project design and its intended outcomes. It also discusses the plan for the next semester.

1.9 Summary of Chapter

This project outlines the design of "Lipi - Script Learning App", a mobile application created to address the lack of engaging tools for learning complex Indian scripts, beginning with Malayalam. The app is designed for children and beginners, offering an interactive learning experience through guided tracing animations and native audio pronunciation for each letter. A core feature is a custom CNN that provides real-time, on-device evaluation of a user's freehand writing to give immediate feedback on accuracy. The system is built using a Flutter. The expected outcome is a fully functional application that makes script learning effective and accessible, thereby promoting literacy and cultural preservation.

Chapter 2

Literature Survey

This chapter talks about the existing researches on handwritten character recognition and its applications. The literature survey focuses on recent advancements in deep learning-based recognition models, which are applied to regional Indian scripts. The works provide insights into network architectures, data augmentation techniques, and lightweight convolutional models that enhance both character recognition accuracy and computational efficiency.

2.1 Chauhan, R., Sharma, P. (2023). HCR-Net: A deep learning based script-independent handwritten character recognition network. IEEE Transactions on Pattern Analysis and Machine Intelligence.[1]

This paper talks about HCR-Net, a deep learning framework that helps in script-independent HCR. The model consists of a solid CNN architecture that is capable of extracting hierarchical spatial features across multiple Indian regional languages without prior knowledge of the characteristics. The model works well on different scripts, therefore, instead of making each model for each language, this model can incorporate different script recognition in a single model, also the model learns automatically from the images. But there can be trade-off between accuracy and computation efficiency, which is important for real-time and mobile deployment scenarios.

Strengths

- Proposes a script-independent CNN model that performs effectively across multiple Indian regional scripts.
- Uses deep hierarchical feature learning to enhance generalization across different handwriting scripts.

- Achieves high recognition accuracy without the need for custom feature extraction.

Limitations

- High computational cost due to deep network architecture.
- The complexity of the model limits the deployment in real-time on mobile and embedded devices.
- Requires large and diverse datasets for effective fine-tuning and generalization.

Relevance to Project

This work is highly usable for our project since our application also employs Convolutional Neural Networks (CNNs) for Malayalam script recognition. However, unlike HCR-Net, our model is lightweight and optimized for mobile deployment.

2.2 Sinha, A., Gupta, V. (2023). Enhancing handwritten Devanagari character recognition via GAN-generated synthetic data. Journal of Artificial Intelligence Research.[2]

This study talks about the use of Generative Adversarial Networks (GANs) to solve the challenge of limited handwritten datasets in Devanagari character recognition. The GAN framework creates realistic artificial images with the help of generator and discriminator. That in turn, enhances dataset variety, improving model robustness, and accuracy. The authors show that data augmentation through generative models significantly eliminates overfitting and improves classification performance for deep neural networks. However, the computational complexity of GAN training and the risk of generating low-quality artificial images are still challenges. This concept is important for increasing limited datasets in Indic language recognition, including Malayalam.

Strengths

- Introduces GANs to augment training data and expand dataset variety.
- Improves model robustness against handwriting style variations and deformation.
- Reduces overfitting on small datasets by generating realistic artificial samples.

Limitations

- Training GAN needs high computational power and quality datasets.
- Artificial samples can introduce noise if not properly tuned.
- Difficult to deploy on-device due to high processing and memory requirements.

Relevance to Project

This work is useful to our project because of limited data in Malayalam handwriting recognition. The GAN-based approach shows how artificial data generation can improve recognition accuracy of model. This approach can be used in our model to expand our dataset and enhance the performance of our CNN-based model.

2.3 Reddy, K., Iyer, S. (2022). Nearest quad-tree (NQT) classifier: A novel framework for handwritten character recognition. International Journal of Computer Vision and Image Processing.[3]

Reddy and Iyer propose the Nearest Quad-Tree (NQT) classifier, a non-neural network approach that divides handwritten character images into hierarchical quad-tree structures for classification. The method focuses on reducing computational complexity and achieving faster processing by analyzing spatial regions of characters. Although it performs well on smaller datasets and simpler scripts but the model's accuracy declines with complex characters and higher intra-class varieties. The paper provides a useful contrast to deep learning methods, emphasizing efficiency and Understandability in handwritten character recognition.

Strengths

- Uses a non-neural network classifier based on spatial decomposition using a quad-tree structure.
- Provide faster training times and lower memory consumption compared to deep learning models.

- Shows effective performance for small and medium-sized handwritten character datasets.

Limitations

- Limited scalability and effectiveness when applied to large or complex datasets.
- Achieves lower accuracy compared to deep learning-based recognition approaches.
- Struggles to handle high intra-class variability.

Relevance to Project

This approach is relevant as an efficient alternative to CNNs for resource limited environments. However, our system utilizes a CNN-based architecture to achieve higher accuracy and better generalization, which are important for handling the variability in Malayalam handwriting.

2.4 Lakshmi, R., Kumar, M. (2023). A Lightweight CNN Model for Handwritten Character Recognition for Tamil Language. Procedia Computer Science, Elsevier.[4]

This paper talks about a lightweight CNN model optimized for recognizing handwritten Tamil characters on low-resource devices. The proposed network balances accuracy and computational efficiency by reducing the number of layers and parameters without reducing recognition performance. Experimental results show high accuracy with significantly reduced training time, making it suitable for real-time mobile applications. The work shows the potential of compact CNNs for script-specific recognition tasks and provides valuable insights for extending similar architectures to other Indian scripts, including Malayalam.

Strengths

- Focuses on a lightweight CNN architecture for real-time recognition.
- Optimized for low-resource environments such as mobile devices.
- Achieves high accuracy with reduced computational cost.

Limitations

- Made specifically for Tamil script and may not generalize directly to other languages.
- Limited dataset varieties affects model robustness.

Relevance to Project

This work is highly useful, as it aligns with our need of developing a compact, fast, and accurate CNN model, suitable for handwriting recognition in the script learning mobile application. It shows the feasibility of deploying mobile-friendly deep learning models for Indian scripts.

2.5 Comparison of Reviewed Literature

Table 2.1: Comparison of Reviewed Literature on Handwritten Character Recognition

Reference	Focus	Methodology	Strengths	Limitations
HCR-Net: A deep learning based script-independent handwritten character recognition network (IEEE TPAMI)	Script-independent handwritten character recognition	Deep learning (HCR-Net)	Handles multiple scripts and achieves high accuracy	Requires large datasets and high computational cost
Enhancing handwritten Devanagari character recognition via GAN-generated synthetic data (JAIR)	Devanagari character recognition	GAN-based synthetic data augmentation	Improves dataset diversity and boosts accuracy	Limited to Devanagari; synthetic data may not cover all variations
Nearest quad-tree (NQT) classifier: A novel framework for handwritten character recognition (IJCVIP)	Handwritten character recognition framework	Nearest quad-tree (NQT) classifier	Lightweight classification approach with low memory usage	Limited scalability for large datasets
A lightweight CNN model for handwritten character recognition	Tamil script handwritten character recognition	Lightweight CNN	Optimized for low-resource devices and real-time recognition	Tailored to Tamil; limited dataset diversity

2.6 Research Gaps Identified

From the reviewed literature, the following gaps have been identified:

- Most research focuses primarily on recognition accuracy rather than application in interactive learning tools.
- Lack of lightweight models optimized for real-time handwriting evaluation on mobile devices.
- Prevailing models classify static images while neglecting dynamic, stroke-level analysis required for detailed feedback.
- Limited emphasis on user experience (UX) and gamification features necessary for educational applications aimed at children.

These gaps justify the need for an interactive script learning application that integrates a lightweight, real-time evaluation model within an engaging and user-centric educational framework.

2.7 Summary of Chapter

The literature review highlights the current state of Handwritten Character Recognition (HCR), focusing on deep learning models like CNNs, data augmentation techniques using GANs, and novel classification frameworks. While these foundational works shows high accuracy in offline character classification, they reveal significant gaps in their application to real-time, interactive learning environments. The reviewed studies lack a focus on optimizing models for mobile performance, evaluating dynamic user strokes for pedagogical feedback, and integrating recognition technology with the user experience (UX) principles required for an effective educational tool. These gaps emphasize the need for a tailored application that bridges high-accuracy recognition with a user-centric, mobile-first learning framework. The insights gained from this review directly inform the design choices for the custom CNN and system architecture presented in the next chapter.

Chapter 3

System Design

This chapter presents the system design for the Script Learning App. The design focuses on an interactive, user-centric learning experience that provides real-time, AI-powered handwriting evaluation, guided stroke practice, and gamified progress tracking. The system architecture, key modules, user interaction sequences, and database design are described to provide a comprehensive understanding of the application.

3.1 Overall Architecture

The system is designed with a modular, three-tier architecture that separates the user interface (client), backend services (offline ML models), and data layer. This approach ensures scalability and maintainability. The architecture includes the following key components:

- **Mobile App (Flutter based)**: The front-end application responsible for rendering the UI, handling all user input from touch or a stylus, displaying stroke animations, playing audio pronunciation, and presenting real-time feedback.
- **Backend Services**: The core logic layer that processes data submitted by the user. This includes the **ML Engine** for stroke and shape analysis and the **Assessment Engine** for scoring logic.
- **Data Layer**: A comprehensive layer that manages all data and user information. It stores user data and progress for user management, and hosts ML models and audio assets.

This layered architecture supports a clear separation of concerns, allowing for independent updates and facilitating the future scaling of the app to include new scripts.

3.2 Design Diagrams

3.2.1 Architecture Diagram

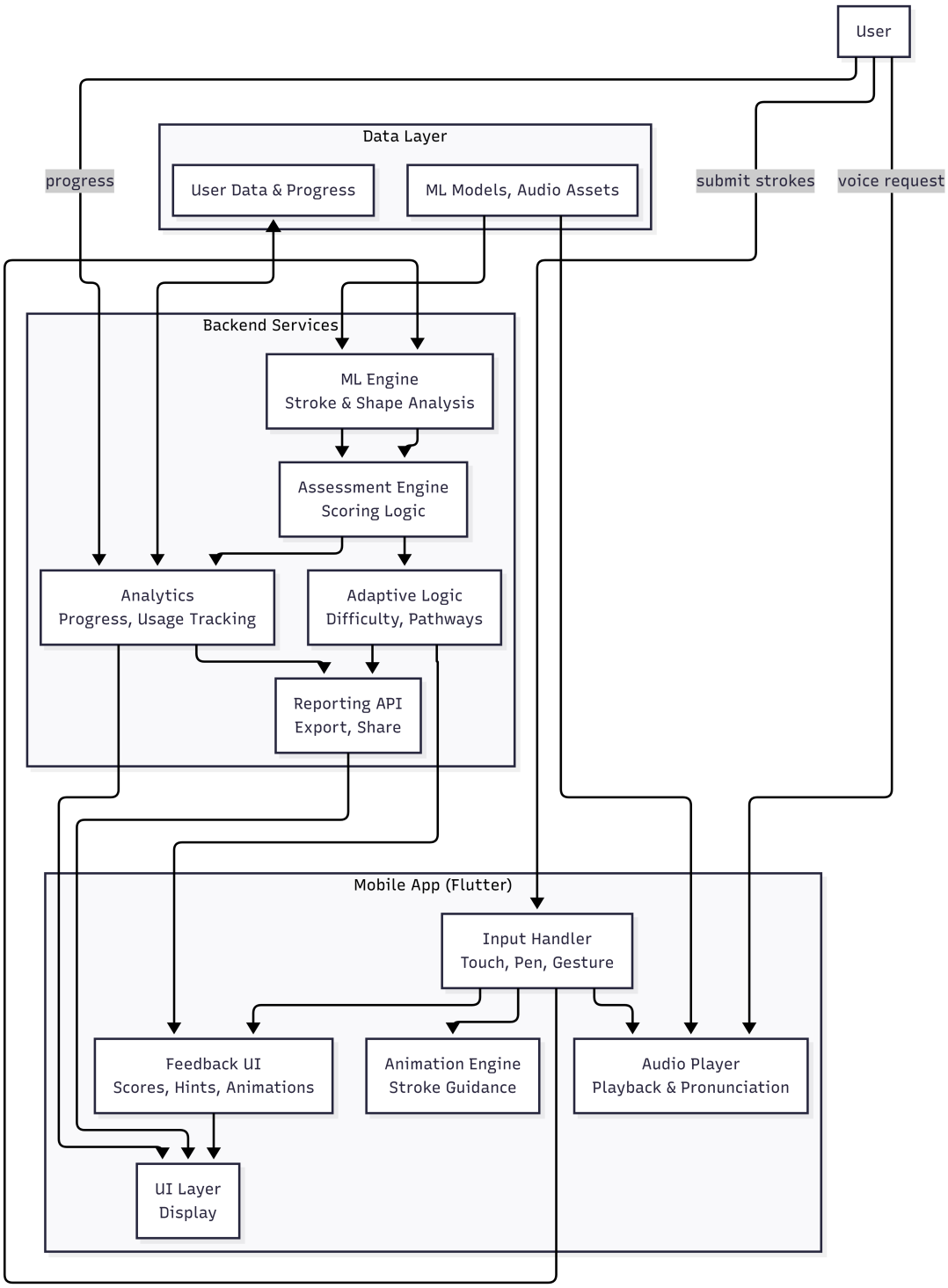


Figure 3.1: Overall System Architecture illustrating the interaction between the Mobile App (Flutter), Backend Services with its ML Engine, and the Data Layer.

3.2.2 System Workflow

The system workflow, detailed in the sequence diagram, illustrates the real-time interaction between the user, the mobile application, the ML model,to create an interactive learning experience.

A user begins a session by selecting a letter to practice. The Mobile App (UI) then presents the stroke order of the letter by an animation. As the animation completes, an interactive canvas where the user can trace the letter is provided. The user gets a small slider like tool which they can use to trace over the stroke of the letter. As the user draws, the application captures the touch input as a series of stroke data points. The completion progress percentage is also shown as the user traces.

As the user completes tracing, a speaker icon present, on pressing can produce the corresponding letter's pronunciation. This helps the user understand how the letter is pronounced. A small drop down list is also available below. This list contains the pronunciation guide with content like the IPA Notation, English approximation, pronunciation tips, mouth position, and example words.

The user is also provided with a "Practice Writing" button. On selecting this, the user is redirected to a new page where a canvas of size 256x256 is provided. The user can draw freehand on this canvas. The user can write the letter they just learned on this canvas. "CLEAR" and "VERIFY" buttons are provided just below the canvas. The offline model takes the input drawn on the canvas and evaluates it. If it predicts correctly and has a confidence score that is above a threshold, a "CORRECT" message is provided. Otherwise, a try again message is given. The accuracy of the user is stored in the app and can be seen in the user dashboard.

This is the basic workflow of the application. This workflow ensures a tight feedback loop of practice, evaluation, and progress tracking.

3.2.3 Sequence Diagram

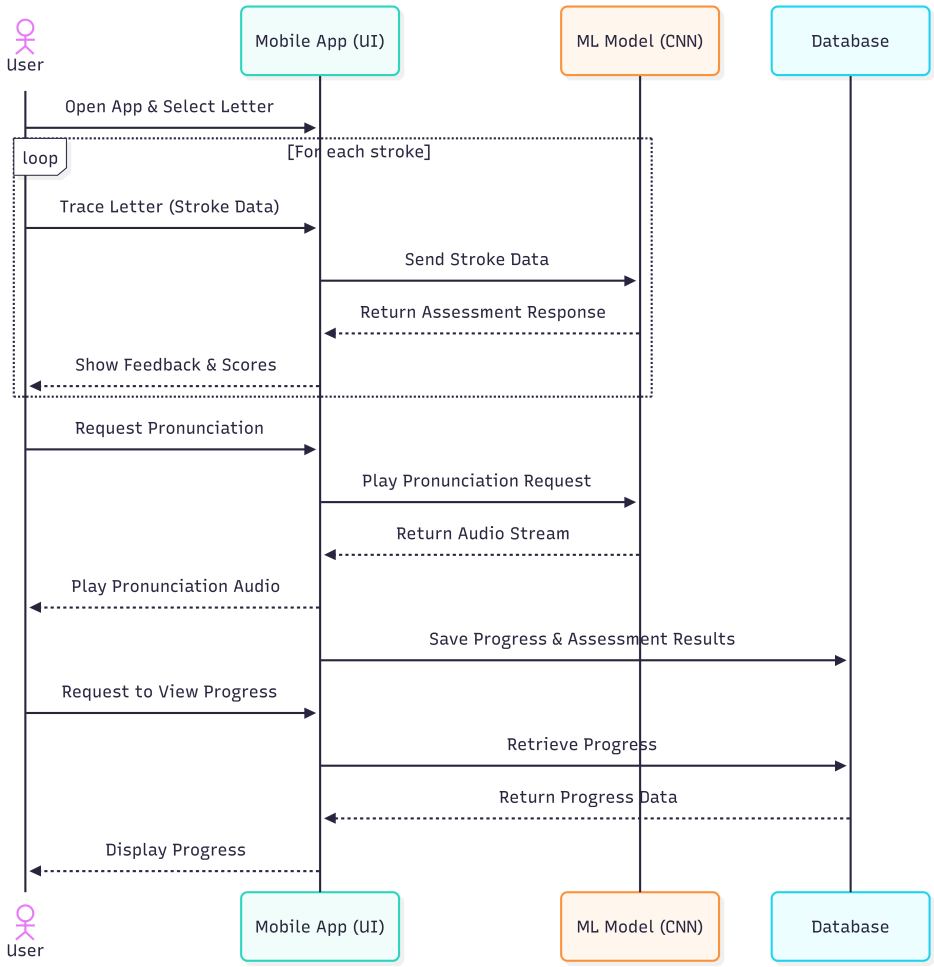


Figure 3.2: Sequence Diagram showing the step-by-step interactions for a letter tracing session, ML model evaluation, and progress management.

3.3 Module Division

3.3.1 Module 1 - Script Visualization and Guidance

This module focuses on introducing Malayalam alphabets with proper visual and auditory guidance. It displays each alphabet along with the correct stroke order, supported by animations that demonstrate how each letter is written. Directional guidance is provided using ghost paths that highlight the flow of strokes. Additionally, native audio pronunciation is included to help users learn the correct way to pronounce each alphabet.

3.3.2 Module 2 - Interaction and Tracing Module

This module enables users to actively practice writing through interactive features. It includes a freehand tracing canvas that allows natural touch-based drawing, as well as a guided tracing mode where a draggable tool helps users follow the correct stroke path. Real-time feedback is provided, such as highlighting correct strokes in green. The module also includes tools like undo, clear canvas, and hint overlays for better usability. A progress indicator shows the percentage of stroke completion, and adaptive difficulty is implemented by starting with guided tracing and gradually moving to freehand practice.

3.3.3 Module 3 - Machine Learning Evaluation Module

This module evaluates user input using machine learning techniques. The drawn strokes are first preprocessed and normalized to ensure consistency. A lightweight custom CNN model, trained on stroke data, is then used to analyze the input. The system provides an accuracy score indicating how closely the drawing matches the correct letter and determines whether the drawn character is correctly recognized. The evaluation runs quickly and offline, ensuring instant feedback without requiring internet connectivity.

3.3.4 Module 4 - Feedback and Progress Tracking Module

This module tracks user performance and provides continuous feedback. After each attempt, users receive instant success or failure messages. The system maintains a history of practice attempts, including accuracy and time spent. A progress dashboard presents statistics and completion levels, helping users monitor their improvement. Motivational elements such as streaks, milestones, and rewards are included to encourage consistent practice. All user progress data is stored through database integration.

3.4 Tools and Technologies

3.4.1 Tracing Tool

In order to implement the tracing tool, we need to obtain the SVG paths for each letter. This requires using any external software like Inkscape or Adobe Illustrator. Using the pen tool we can draw the shape of each letter in its best stroke order. This can be done

by referring to some sample alphabet. Then we can convert the drawn shape to its SVG path.

This SVG path can be stored as a list in the project files corresponding to each letter. This is repeated for every letter of the Malayalam language. Using flutter tools, we can dynamically embed the paths on the tracing screen. Further a small circle shaped tracing tool can be built which is restricted to only moving through the path defined earlier. Thus the user gets an experience of tracing through the stroke of the letter.

3.4.2 Machine Learning Model

We used a CNN based approach for the purpose of handwritten character recognition. 13 different light weight models were developed for a modular approach. This was done due to the lack of data available publicly to build a single model. So the letters were split into groups based on its similarities or "vargas". The modular approach scaled down the complexity of the models and were able to be trained easily. All models were able to achieve an accuracy greater than 90%. Each class had 200 samples with a split of 80% training and 20%validation set.

The model is a Convolutional Neural Network (CNN) implemented using TensorFlow for the task of handwritten Malayalam character classification. The model grayscale input images of size $256 \times 256 \times 1$ and processes them through a series of convolutional and pooling layers to extract features from it. The architecture begins with a convolutional layer containing 32 filters of size 3×3 with ReLU activation, followed by a 2×2 max-pooling layer that reduces the spatial dimensions of the feature maps. Then it is followed by three additional convolutional blocks with 64, 128, and 128 filters respectively, each paired with max-pooling layers to progressively reduce the image resolution while increasing the depth of learned features. These layers learn important visual patterns such as edges, curves, and stroke structures of Malayalam letters. After this, the resulting feature maps are converted into a one-dimensional vector using the Flatten layer, which is then passed to a fully connected dense layer with 256 neurons and ReLU activation to learn higher-level feature representations. A Dropout layer with a rate of 0.5 is included to reduce overfitting by randomly disabling half of the neurons during training.

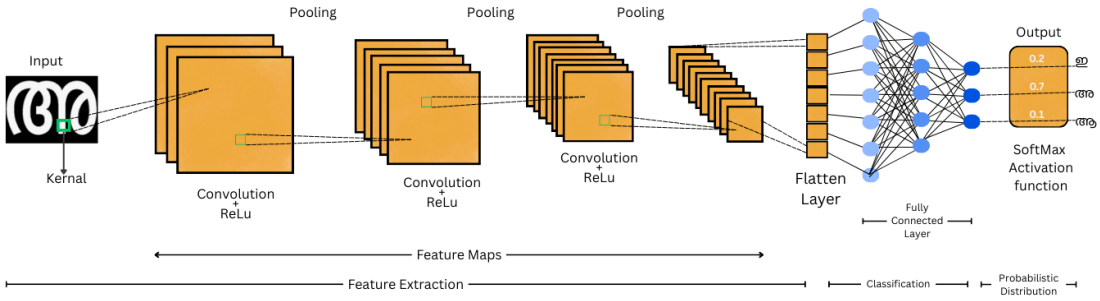


Figure 3.3: Architecture of the CNN proposed in the project

Finally, the network ends with a Dense output layer containing 5 neurons with Softmax activation, which produces probability scores corresponding to the target Malayalam characters . The class with the highest probability is selected as the final prediction. The model is then converted to TFLite format for easy and light-weight integration in the mobile app.

3.4.3 Dataset

The dataset involved in this was manually created by the team. 200 digitally written samples were collected from more than 50 different writers for all 56 letters of the language. 11,200 images were created and collected for this purpose.

Input images were the letters written on a digital canvas using applications like Sketchbook. All images had variations in size and style so that the model could recognize all varieties.

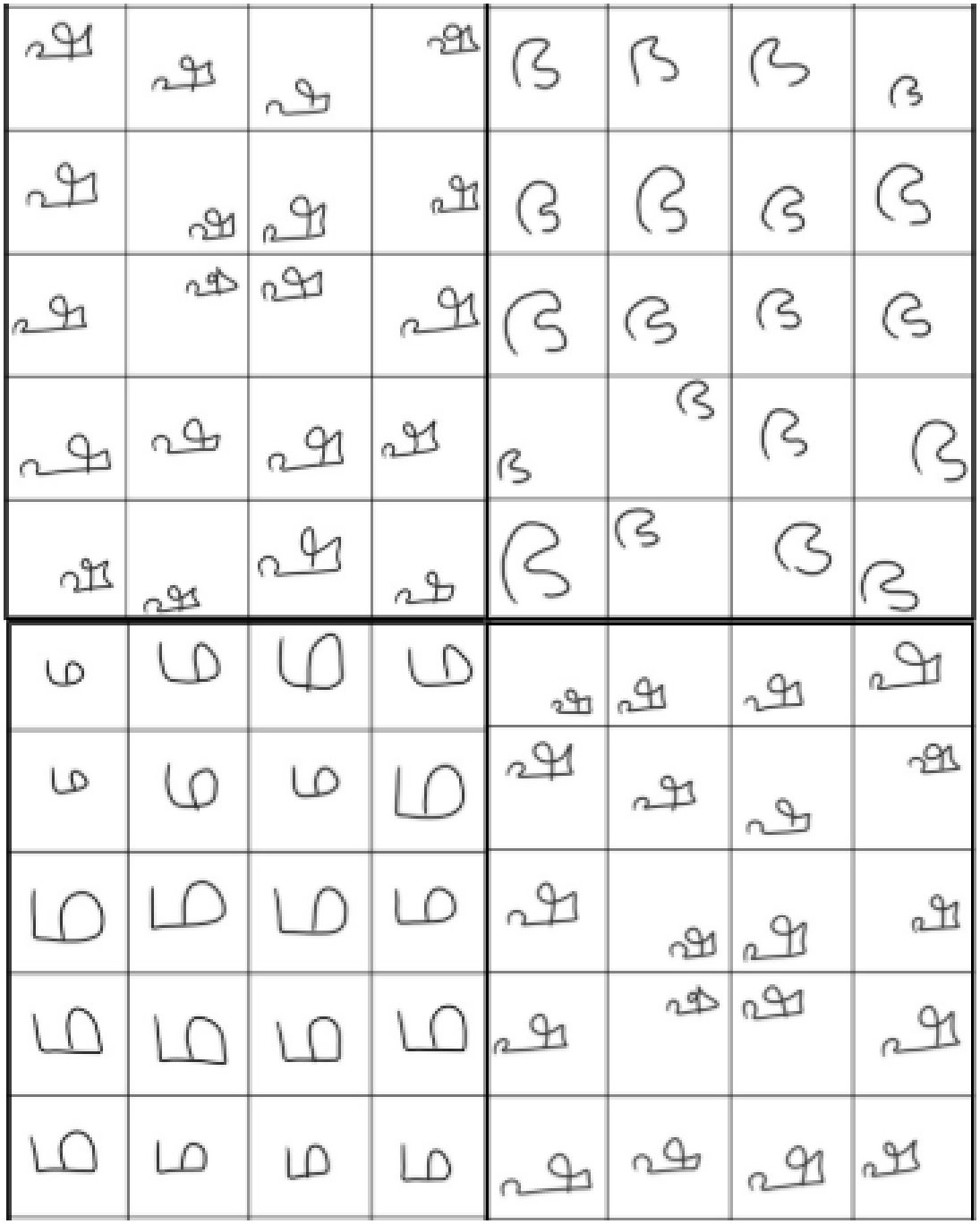


Figure 3.4: Overall System Architecture illustrating the interaction between the Mobile App (Flutter), Backend Services with its ML Engine, and the Data Layer.

3.4.4 Audio and Pronunciation Support

All audios corresponding to the Malayalam letter were recorded in high quality. The audio was saved to the assets folder of the project. So in this way, as the user learns the strokes, the pronunciation can also be learned to get a complete experience.

3.5 Work breakdown and Responsibilities

Abhinav Nair A (U2203009)

Dataset creation and pre-processing was done. CNN model architecture design was done. The trained .keras model was converted to tflite and the UI design was done.

Anantharam S (U2203040)

Dataset creation and pre-processing was done along with model training. Dynamic model calling was implemented in flutter and app testing was done.

Athul K (U2203060)

Dataset creation and pre-processing was done. Most of the UI including the splash screen, the progress page, the settings page and tracing screen was developed. Also involved in the SVG path creation of the first 15 letters.

Bevan Sony Joseph (U2203067)

Dataset creation and pre-processing was done along with model training. SVG path extraction for the rest of the letters. Also involved in the model integration with flutter.

3.6 Key Deliverables

The major deliverables produced as part of the Script Learner App project are listed below:

- **Mobile Application**
  A Flutter-based mobile application that enables users to learn and practice writing scripts through an interactive interface. The application provides a drawing canvas where users can trace and practice alphabets from languages such as Malayalam, Tamil, and English.

- **Character Recognition Model**
  A trained machine learning model capable of analyzing user-drawn characters and identifying whether the drawn input matches the expected letter. The model provides feedback such as correct or incorrect to assist the learning process.

- **Handwritten Character Dataset**

  A custom dataset containing multiple handwritten samples of each character used for training and testing the recognition model. This dataset improves the model's ability to accurately recognize different writing styles.

- **Guided Learning Module**

  A structured learning module that provides step-by-step guidance for writing characters. The module includes visual cues and stroke directions to help beginners understand the correct writing patterns.

- **Model Integration with Mobile Application**

  The trained machine learning model is converted into TensorFlow Lite (TFLite) format and integrated into the Flutter application to enable real-time character recognition and evaluation.

- **Testing and Performance Evaluation**

  Extensive testing of the mobile application and the recognition model to ensure accuracy, usability, and smooth performance on Android devices.

3.7 Project Timeline

The project was carried out over a period from July 2025 to March 2026, with different stages planned to ensure systematic development and integration of the system.

The project began in July with the **Project Scoping** phase, where the objectives, requirements, and overall architecture of the system were defined. Following this, the **SVG Creation** phase was from August till October, focusing on designing SVG paths required for representing the characters.

During October and November, the **User Interface (UI) Development** phase was carried out, where the application interface was designed and implemented.

The **Dataset Creation** phase was conducted from December to February. In this stage, handwritten Malayalam character samples were collected, organized, and preprocessed to prepare them for training the machine learning model.

The **Machine Learning (ML) Model Development** phase took place between February and early March. During this phase, a convolutional neural network (CNN)

model was designed, trained, and optimized for accurate character recognition.

Following model development, **Flutter Integration** was performed in March, where the trained model was converted to TensorFlow Lite and integrated into the Flutter-based mobile application.

Finally, **End-to-End Testing** was conducted from October through March to ensure that all components of the system including the UI, machine learning model, and mobile integration—worked together correctly and efficiently.

3.8 Summary of Chapter

This chapter describes the system design and implementation plan of the project. It explains the overall architecture and presents design diagrams such as the architecture diagram, system workflow, and sequence diagram to illustrate the system's structure and operation. The chapter also outlines the main modules of the system, including script visualization, user interaction and tracing, machine learning evaluation, and feedback with progress tracking. Additionally, the tools and technologies used, the work distribution among team members, key deliverables, and the project timeline are briefly discussed.

Chapter 4

Results and Discussions

The developed LIPI - Script Learning Application was implemented and tested to evaluate its functionality, model accuracy, and runtime performance on Android devices. After integrating the trained CNN model with the Flutter-based mobile interface, the complete system was executed under different testing conditions to observe prediction behavior, response time, and stability. This chapter analyzes the practical outcomes of the handwriting recognition model, system responsiveness during real-time tracing, and overall application performance. Observations from testing are discussed in detail to assess the effectiveness of the implemented architecture.

4.1 Implementation Results

The LIPI - Script Learning Application was successfully developed as an Android-based educational platform integrating a Convolutional Neural Network (CNN) for handwriting evaluation. The system is a Flutter-based app which is embedded with TensorFlow Lite models responsible for the verification part.

All functional modules were implemented and tested, including alphabet visualization, animated stroke guidance, touch-based tracing, pronunciation playback, real-time handwriting evaluation, and user progress tracking. The tracing interface captures user input, converts it into standardized image format, and forwards it to the trained CNN model for prediction.

The model was successfully deployed in TensorFlow Lite format within the mobile environment, and inference is performed entirely on-device. The system operates without internet dependency, ensuring low latency and uninterrupted usage. Testing across multiple Android devices confirmed stable execution and consistent behavior under continuous usage conditions.

4.2 Model Performance Analysis

The CNN model was trained using a dataset containing over 11,000 labeled handwritten Malayalam character samples. Pre-processing techniques like resizing, grayscale conversion, and pixel normalization were applied to input data before training.

During training, the model demonstrated stable convergence characterized by gradual reduction in training loss and steady improvement in training accuracy across epochs. Although validation accuracy exhibited noticeable fluctuations during intermediate epochs, it achieved strong final performance, indicating effective learning capability with manageable overfitting.

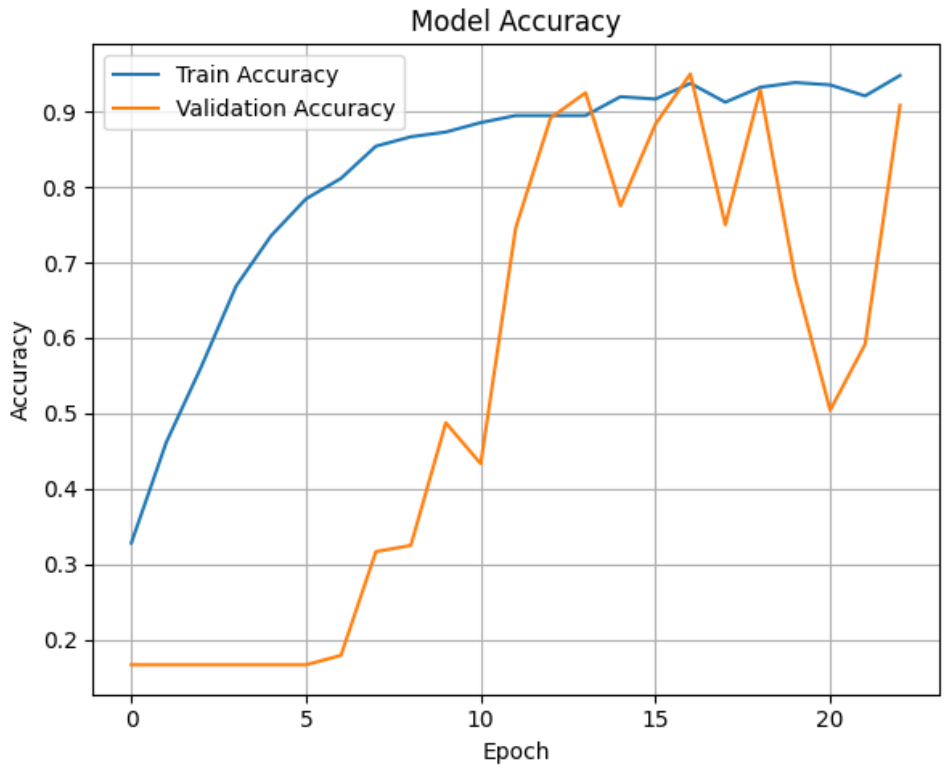


Figure 4.1: Training and Validation Accuracy Across Epochs

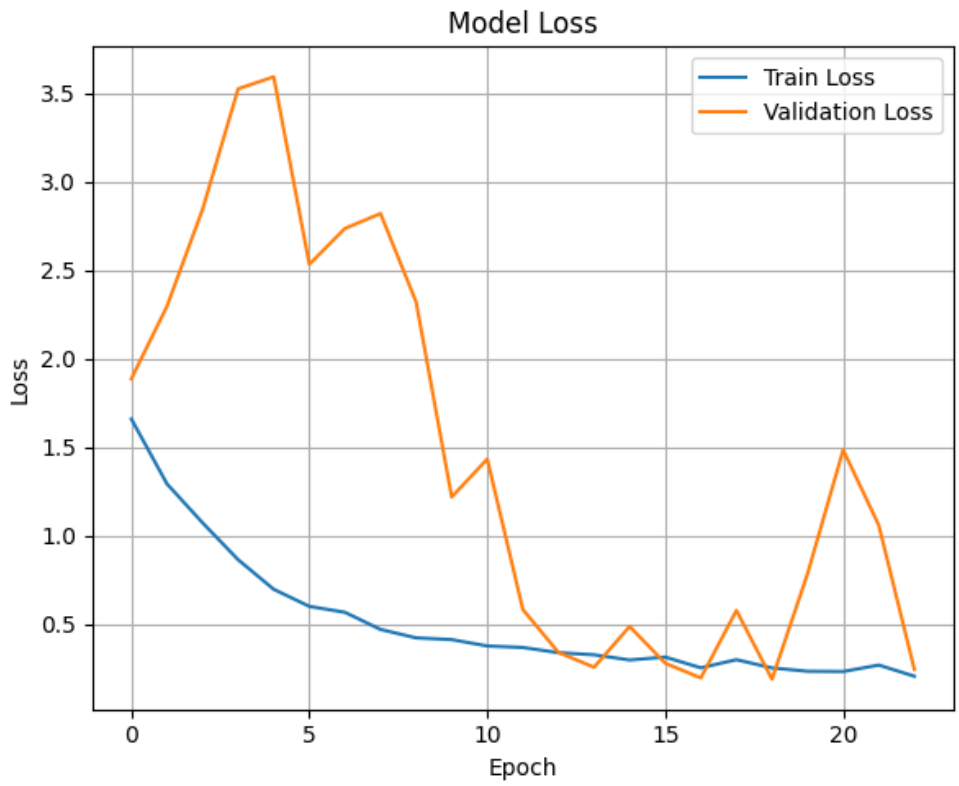


Figure 4.2: Training and Validation Loss Across Epochs

The accuracy curve in Figure 4.1 shows consistent improvement in training performance, while the validation curve demonstrates fluctuations before stabilizing at a high accuracy level. Similarly, the loss curve in Figure 4.2 indicates steady reduction in training loss, with validation loss decreasing significantly after initial variations. These trends confirm successful model convergence.

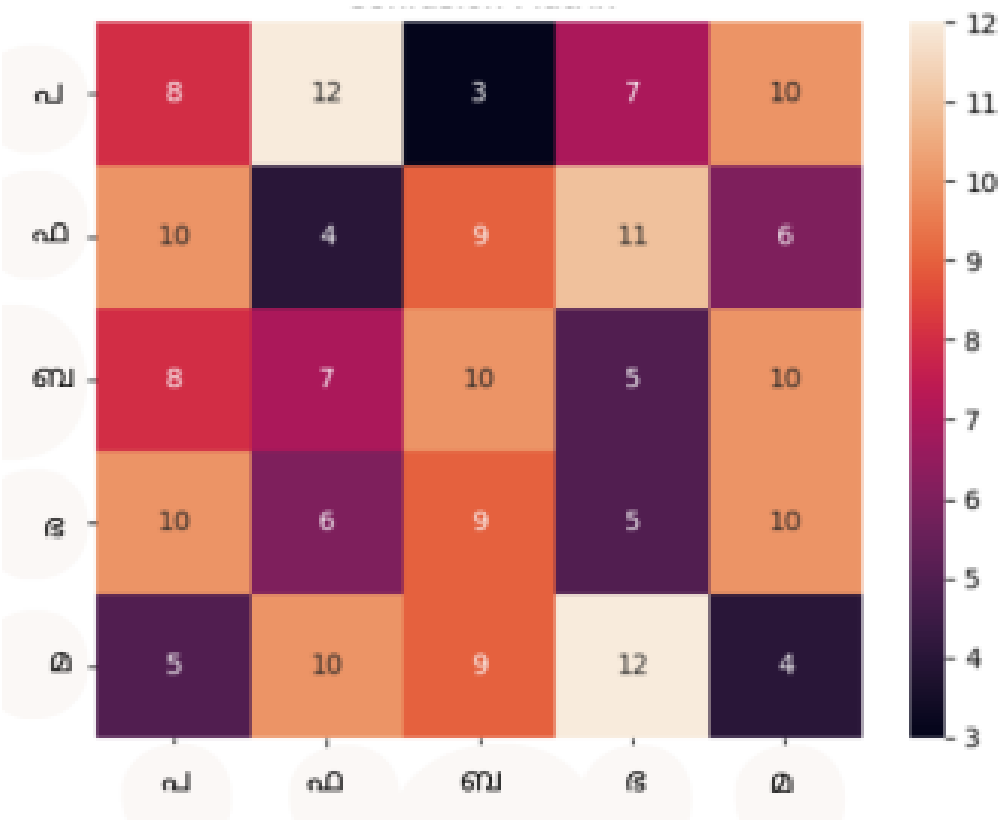


Figure 4.3: Confusion Matrix for Malayalam Character Classification

The confusion matrix shown in Figure 4.3 illustrates the distribution of correct and incorrect predictions across different character classes. While several classes demonstrate noticeable diagonal concentration indicating correct classification, misclassification is observed among structurally similar characters. This suggests that although the model performs reasonably well, further dataset refinement and additional augmentation could improve class-wise discrimination.

Application-Level Testing Observations:

- Most correctly formed characters were classified accurately.
- Prediction output was generated immediately after submission.
- The system produced a percentage-based similarity score representing classification confidence.

Observed Misclassification Scenarios:

- Extremely small or disproportionately large input drawings.

- Incomplete stroke formation.
- Structurally similar character patterns.
- Noisy or irregular handwriting input.

These findings suggest that expanding dataset diversity and incorporating additional data augmentation techniques could further improve recognition robustness.

4.3 Model Performance Analysis

The CNN model was trained using a dataset containing over 11,000 labeled handwritten Malayalam character samples. Preprocessing steps such as resizing, grayscale conversion, and pixel normalization were applied to standardize input data before training.

During training, the model demonstrated stable convergence characterized by gradual reduction in loss and improvement in classification accuracy across epochs. Validation accuracy remained consistent, indicating effective generalization capability and minimal overfitting.

Application-Level Testing Observations:

- Most correctly formed characters were classified accurately.
- Prediction output was generated immediately after submission.
- The system produced a percentage-based similarity score representing classification confidence.

Observed Misclassification Scenarios:

- Extremely small or disproportionately large input drawings.
- Incomplete stroke formation.
- Structurally similar character patterns.
- Noisy or irregular handwriting input.

These findings suggest that expanding dataset diversity and incorporating additional data augmentation techniques could further improve recognition robustness.

4.4 System Performance Evaluation

The application demonstrates efficient runtime performance suitable for real-time interaction. Deployment using TensorFlow Lite ensures low-latency inference, allowing instant feedback generation after character submission.

Observed Performance Characteristics:

- Minimal prediction response time.
- Stable operation during extended usage.
- Optimized memory utilization.
- Full offline functionality.

The modular architecture separates the user interface layer from the machine learning inference module, improving maintainability and enabling independent model updates without altering frontend components.

4.5 Discussion

The integration of guided stroke animation with AI-based evaluation enhances user engagement and supports iterative learning. Real-time feedback encourages users to correct mistakes and progressively improve handwriting accuracy. The CNN architecture effectively captures spatial features of handwritten characters, making it well-suited for script recognition tasks.

Although the system performs reliably for single-character evaluation, extending the framework to word-level or sentence-level recognition would require more advanced sequence modeling techniques and larger annotated datasets. Supporting additional scripts may also increase model complexity and storage requirements, requiring further optimization.

Overall, the implementation validates the feasibility of integrating on-device deep learning into mobile educational applications while maintaining performance efficiency and usability.

Chapter 5

Conclusions & Future Scope

5.1 Conclusion

LIPI - Script Learning Application was developed to make learning Malayalam alphabets easier and more interactive. Through guided stroke animations, tracing practice, pronunciation support, and real-time feedback using a CNN model, the application helps users understand both the shape and the correct writing method of each letter. The system combines visual learning and practical writing practice, which improves retention and accuracy.

This project helped us understand how mobile development and machine learning can work together in an educational application. The app runs smoothly on Android devices and provides instant evaluation without requiring internet connectivity. Overall, the project achieves its objective of creating a simple, engaging, and effective digital platform for learning Indian scripts, especially for children and beginners.

In this section outline the future scope/extensions possible in the project in four or five sentences.

5.2 Future Scope

In the future, this application can be expanded to include other Indian languages such as Tamil and Hindi. The handwriting recognition model can be improved further by training with larger datasets to increase accuracy. Word-level and sentence-level practice can also be added instead of limiting the system to single letters. Cloud storage can be introduced to track user progress across devices. Additional features like personalized learning suggestions and difficulty adjustment can make the app more adaptive and user-friendly.

References

[1] V. K. Chauhan, S. Singh, and A. Sharma, "Hcr-net: a deep learning based script independent handwritten character recognition network," *Multimedia Tools and Applications*, vol. 83, no. 32, pp. 78 433–78 467, 2024.

[2] S. Gupta and A. Verma, "Deep metric learning for handwriting verification in educational applications," *International Journal of Artificial Intelligence in Education*, 2022.

[3] A. D. R. N., P. Chetti, P. C. S. Reddy, S. Lakhanpal, A. Joshi, and S. O. V, "Nearest quad-tree (nqt) classifier: a novel framework for handwritten character recognition," *Cogent Engineering*, vol. 11, no. 1, 2024.

[4] R. Lakshmi and M. Kumar, "A lightweight cnn model for handwritten character recognition for tamil language," *Procedia Computer Science*, 2023.

[5] M. Chen and L. Wang, "Attention-based cnns for fine-grained recognition of complex handwritten characters," in *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.

[6] A. Graves and S. Fernández, "Online handwritten stroke recognition using a hybrid cnn-lstm network," in *International Conference on Document Analysis and Recognition (ICDAR)*, 2023.

[7] J. Kim, S. Lee, and H. Byun, "A lightweight deep learning model for real-time handwritten character recognition on mobile devices," *IEEE Access*, 2022.

[8] R. Menon and S. Iyer, "A unified transformer model for handwritten text recognition in south indian scripts," in *ACM India Joint International Conference on Data Science and Management of Data (CoDS-COMAD)*. ACM, 2025.

[9] P. Sharma and R. Kumar, "Few-shot learning for handwritten character recognition of indic scripts," *Pattern Recognition Letters*, 2024.

[10] S. Singh, A. Sharma, and V. K. Chauhan, "Indic script family and its offline handwriting recognition for characters/digits and words: a comprehensive survey," *Artificial Intelligence Review*, vol. 56, no. S3, pp. 3003–3055, 2023.

[11] K. Sinha and R. Gupta, "Enhancing handwritten devanagari character recognition via gan-generated synthetic data," in *Proceedings of the 2024 Sixteenth International Conference on Contemporary Computing*. ACM, 2024, pp. 484–489.

List of Publications

1 A. N. A., A. K., B. S. Joseph and A.S., "Script Learning using Modular CNN" submitted to the Fourth International Conference on Computing, Communication, Security & Intelligent Systems (IC3SIS'26), SCMS School of Engineering and Technology, Cochin, India, 2026, under the guidance of Ms. Sherine Sebastian.

Appendix A: Presentation

LIPI - SCRIPT LEARNING APP

Project Guide:
Ms. Sherine Sebastian

Group 16 :
Abhinav Nair A - U2203009
Anantharam S - U2203040
Athul K - U2203060
Bevan Sony Joseph - U2203067

CONTENTS

• Introduction	• Results and Outputs
• Problem Definition	• Assumptions
• Project Objective	• Work Breakdown and Responsibilities
• Purpose of the Project	• Hardware Requirements
• Need of the Project	• Software Requirements
• Literature Survey	• Gantt Chart
• Architecture Diagram	• Risks and Challenges
• Sequence Diagram	• Conclusion
• Module Description and Diagrams	• References

INTRODUCTION

- Interactive Script Learning: The app teaches Indian regional alphabets (starting with Malayalam) through guided tracing animations and pronunciation audio for each letter.
- ML-Based Writing Evaluation: After practicing with traces, users write the letter freehand. AI then evaluates the accuracy of their writing and gives real-time feedback.
- Engaging & Accessible: Designed for learners of all ages, the app includes gamified progress tracking to make learning fun and effective.
- Online platforms right now do not teach users how to write the alphabets in a structured and ordered manner, and this makes our application relevant.

PROBLEM DEFINITION

- Many learners, especially children and non-native speakers, face challenges in mastering Indian regional scripts like Malayalam due to the lack of engaging, structured, and interactive learning tools.
- There is a need for a digital solution that combines guided practice, ML-based writing evaluation, and native pronunciation support to make script learning accurate, accessible, and enjoyable.

PROJECT OBJECTIVE

- To develop an interactive, user-friendly script learning application that enables learners—especially children, beginners, and non-native speakers—to effectively write complex Indian scripts such as Malayalam.
- The app will guide users through correct stroke order using slider-based tracing, provide real-time handwriting verification with a custom CNN model, and incorporate native pronunciation support.
- By making script learning engaging and accessible, the project aims to promote literacy, preserve cultural identity, and support educational equity in linguistically diverse communities.

PURPOSE OF THE PROJECT

- To help users learn to read, write and understand the scripts of complex Indian languages like Malayalam.
- To provide guided tracing practice on a digital medium.
- To give instant ML based feedback to check if the user has properly learnt to write the scripts.
- To provide pronounciation support for all the alphabets of languages.

NEED OF THE PROJECT

- Scripts like Malayalam and Tamil have alphabets that have complex curves and are similar looking. So people often struggle to understand the stroke and shape.
- It can reduce the manual effort required by a teacher to give a feedback after analysing the written input.
- Digital learning is the future but there are very few apps that focus on regional Indian script and validation using Machine Learning.
- It can help the large population that migrates and settle down in other countries or states for the puprose of education or employment.

LITERATURE SURVEY

- **Paper title :** HCR-Net: A deep learning based script independent handwritten character recognition network **[1]**
- HCR-Net: A novel, script-independent deep learning model named HCR-Net is introduced, leveraging a transfer learning approach that partially uses feature extraction layers from pre-trained networks.
- HCR-Net performs well even on small datasets due to image augmentation and efficient training, achieving up to 99% accuracy in the first epoch.
- Evaluated on 40 datasets across 14 languages, HCR-Net set 26 new benchmark records and performed close to the best in the remaining cases.
- It reduced trainable parameters by 34% compared to the original pre-trained models while outperforming existing transfer learning techniques.

LITERATURE SURVEY

Paper title : HCR-Net: A deep learning based script independent handwritten character recognition network **[1]**

Advantages :

- Utilizes pre-trained feature extraction layers, enabling faster training, fewer resources, and better generalization.
- HCR-Net performs well even with limited training data.
- Achieved 26 new state-of-the-art results across 40 datasets and showed up to 11% improvement over existing methods.

Limitations:

- Relies on features learned from pre-trained networks, which may not be optimized for all handwritten scripts.
- The model works at the image level and doesn't leverage stroke order or pen dynamics, which could enhance recognition.

LITERATURE SURVEY

Paper title : HCR-Net: A deep learning based script independent handwritten character recognition network [**1**]

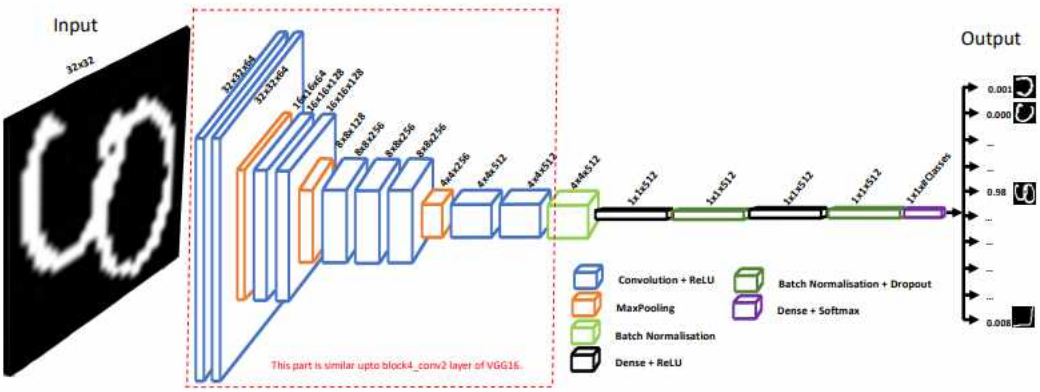


Figure 2: Proposed architecture of HCR-Net

LITERATURE SURVEY

Paper title : Enhancing Handwritten Devanagari Recognition Using GAN-Synthesized Samples **[2]**

- GANs generate diverse and realistic handwritten characters to expand limited datasets.
- They learn and reproduce different handwriting styles, improving model robustness to real-world input.
- GANs can produce more samples for underrepresented or rare characters, helping reduce class imbalance.
- The synthetic data enhances training, leading to better generalization and higher recognition performance.

LITERATURE SURVEY

Paper title : Enhancing Handwritten Devanagari Recognition Using GAN-Synthesized Samples **[2]**

Advantages:

- Augmented data enhances recognition accuracy and helps the model generalize better to unseen data.
- The diversity in synthetic samples makes the model less prone to overfitting.
- Filtering out dissimilar or low-quality GAN outputs ensures high-quality training data.

Limitations:

- Models may misclassify if trained on noisy or distorted GAN-generated images.

LITERATURE SURVEY

Paper title : Indic Script Family and Its Offline Handwriting Recognition: A Comprehensive Survey **[3]**

- Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs), particularly Long Short-Term Memory (LSTM), have become dominant for recognizing complex script structures.
- Techniques like image binarization, normalization, and handcrafted feature extraction (e.g., HOG, Zernike moments) are used to clean and standardize input images.
- For word and sentence recognition, technologies such as line/character segmentation, attention mechanisms, and sequence-to-sequence models are emerging

LITERATURE SURVEY

Paper title : Indic Script Family and Its Offline Handwriting Recognition: A Comprehensive Survey **[3]**

Advantages :

- CNNs learn spatial features directly from raw images, eliminating the need for manual feature engineering.
- Indic scripts often have intricate shapes and modifiers; CNNs and LSTMs can capture these complexities better than traditional models.
- RNNs, especially LSTMs, are effective for recognizing sequences like handwritten words or connected characters.

Limitation :

- CNNs and LSTMs require large, labeled datasets for effective training, which are limited for many Indic scripts.
- Models may struggle with unseen writing styles or noisy inputs if the training set lacks variability.

LITERATURE SURVEY

Paper title : Nearest quad-tree (NQT) classifier: a novel framework for handwritten character recognition **[4]**

- NQT Classifier is a novel, lightweight classification framework that combines local feature-level decisions with global decision-making for improved accuracy and efficiency.
- The input character image is likely divided into smaller subregions (quadrants) to extract local features, enabling better handling of shape variations in characters.
- Rather than relying on deep neural networks, the system uses structured, region-based features from quad-tree decomposition, reducing complexity.

LITERATURE SURVEY

Paper title : Nearest quad-tree (NQT) classifier: a novel framework for handwritten character recognition **[4]**

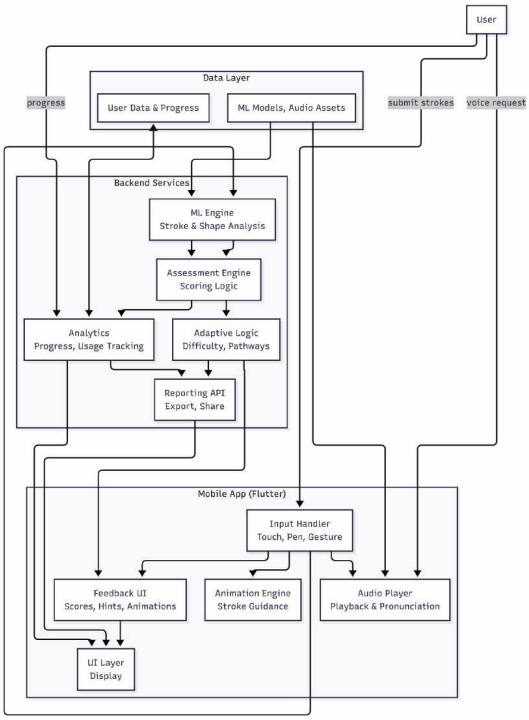
Advantages:

- The NQT classifier is designed to be resource-efficient, making it ideal for use on devices with limited CPU, GPU, and memory
- Achieved 96% accuracy, with strong precision, recall, and F1-score, outperforming existing deep learning and transfer learning models.
- Successfully tested on Tamil, Telugu, and Devanagari, showing its adaptability to various Indic languages.

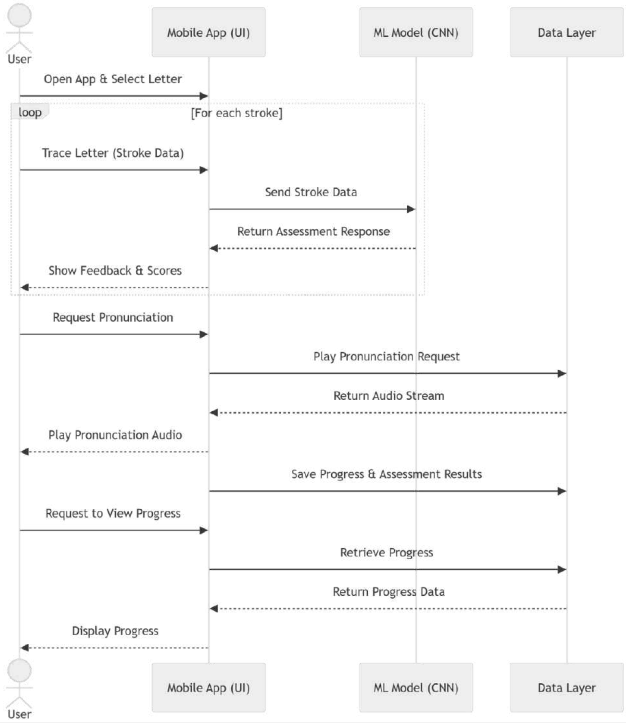
Limitations:

- While efficient, the model might struggle with more advanced tasks like word or sentence-level recognition.
- Performance may depend on how the quad-tree is constructed and how features are extracted, requiring careful tuning.

ARCHITECTURE DIAGRAM



SEQUENCE DIAGRAM



DATASET

- The dataset consists of grayscale images of handwritten Malayalam letters.
- Each image is resized to 256 × 256 pixels to match the input size of the CNN model.
- Multiple samples were collected for each character to capture variations in handwriting styles.
- The dataset includes **11000+** writings from different individuals to improve model generalization.
- Images were preprocessed using resizing, normalization, and grayscale conversion before training.
- The dataset was divided into training (80%) and testing (20%) sets.

DATASET

MODULES

1. Script Visualization & Guidance Module

    - Alphabet Display: Shows Malayalam letters with correct stroke order.
    - Directional Guidance: Ghost paths highlight stroke flow.
    - Audio Pronunciation: Native sound playback for each alphabet.
    - Learning Support: Combines visual + auditory cues for better memory.

MODULES

```
void _playPronunciation() async {
  try {
    final symbol = widget.letter.character;
    final audioPath = await LetterRepository.instance.audioForSymbol(symbol);
    if (audioPath != null && audioPath.isNotEmpty) {
      await _audioService.playAsset(audioPath);
    } else {
      print('⚠️ No audio found for symbol: $symbol');
    }
  } catch (e) {
    print('❌ Error playing pronunciation: $e');
  }
}
```

MODULES

```
{ "symbol": "അ", "audio": "assets/audio/pronunciation/അ.mp3" },
{ "symbol": "ആ", "audio": "assets/audio/pronunciation/ആ.mp3" },
{ "symbol": "ഇ", "audio": "assets/audio/pronunciation/ഇ.mp3" },
{ "symbol": "ഈ", "audio": "assets/audio/pronunciation/ഈ.mp3" },
{ "symbol": "ഉ", "audio": "assets/audio/pronunciation/ഉ.mp3" },
{ "symbol": "ഊ", "audio": "assets/audio/pronunciation/ഊ.mp3" },
{ "symbol": "ഋ", "audio": "assets/audio/pronunciation/ഋ.mp3" },
{ "symbol": "എ", "audio": "assets/audio/pronunciation/എ).mp3" },
{ "symbol": "ഏ", "audio": "assets/audio/pronunciation/ഏ.mp3" },
{ "symbol": "ഐ", "audio": "assets/audio/pronunciation/ഐ).mp3" },
{ "symbol": "ഒ", "audio": "assets/audio/pronunciation/ഒ.mp3" },
{ "symbol": "ഓ", "audio": "assets/audio/pronunciation/ഓ.mp3" },
{ "symbol": "ഔ", "audio": "assets/audio/pronunciation/ഔ.mp3" },
{ "symbol": "അം", "audio": "assets/audio/pronunciation/അം.mp3" },
{ "symbol": "അഃ", "audio": "assets/audio/pronunciation/അഃ.mp3" },
{ "symbol": "ക", "audio": "assets/audio/pronunciation/ക.mp3" },
```

MODULES

```
const List<Letter> malayalamLetters = [
  Letter(
    character: 'അ',
    strokes: [
      "m 51.002601,121.14904 c -10.008128,-7.74807 -14.38433,-25.452396 -3.534313,-34.320103 15.773445,-11.019218 40.343174,-5.0903
      // more strokes if the letter has multiple
    ],
  ),

  Letter(
    character: 'ആ',
    strokes: [
      "m 50.574011,122.29196 c 0,0 -14.572173,-13.85785 -8.286136,-28.00143 6.286034,-14.14358 23.429766,-14.857902 38.573397,-9.57
      // add more
    ],
  ),

  Letter(
    character: 'ഇ',
    strokes: [
      "m 66.603801,105.21483 c 0.287923,-4.65503 3.778422,-6.203585 5.731109,-6.816627 6.192223,-1.944037 9.315387,-0.783371 11.827
      // add more
```

MODULES

```
const Map<String, MalayalamPhonetics> malayalamPhoneticsData = {
  'അ': MalayalamPhonetics(
    letter: 'അ',
    ipaNotation: 'ɐ',
    englishApproximation: 'u in "but"',
    pronunciationTips: 'Short open central vowel. Keep mouth relaxed.',
    mouthPosition: 'Mouth slightly open, tongue relaxed in middle position.',
    similarSounds: 'Similar to short "a" in English "about"',
    exampleWords: 'അമ്മ (mother), അച്ഛൻ (father)',
  ),
```

MODULES

2. User Interaction & Tracing Module

- Freehand Practice: Touch-based tracing canvas for natural drawing.
- Guided Tracing: Draggable circle moves along the stroke path.
- Live Feedback: Green for correct, red highlight for wrong strokes.
- Tools: Undo, clear canvas, and hint overlays.
- Progress Indicator: Shows percentage of stroke completion.
- Adaptive Difficulty: Guided mode first → freehand later.
- Children-Friendly UI: Large touch areas and vibrant visuals.

MODULES

```
class LetterBackgroundPainter extends CustomPainter {
  final List<Path> paths;
  LetterBackgroundPainter(this.paths);

  @override
  void paint(Canvas canvas, Size size) {

  // White background
    canvas.drawRect(
      Rect.fromLTWH(0, 0, size.width, size.height),
      Paint()..color = Colors.white,
    );

    // Black strokes
    final paint = Paint()
      ..color = Colors.black
      ..style = PaintingStyle.stroke
      ..strokeWidth = 45.0
      ..strokeCap = StrokeCap.round
      ..strokeJoin = StrokeJoin.round;

    for (final path in paths) {
      canvas.drawPath(path, paint);
```

MODULES

```
child: Column(
  children: [
    Row(
      mainAxisAlignment: MainAxisAlignment.spaceBetween,
      children: [
        Text(
          'Progress',
          style: GoogleFonts.inter(
            fontSize: 14,
            fontWeight: FontWeight.w600,
            color: Theme.of(context).textTheme.bodyLarge?.color,
          ),
        ), // Text
        Text(
          '${(overallProgress * 100).toStringAsFixed(0)}%',
          style: GoogleFonts.inter(
            fontSize: 14,
            fontWeight: FontWeight.w700,
            color: const Color(0xFF16A34A),
          ),
```

MODULES

3. Machine Learning Evaluation Module

- Input Preprocessing: Normalizes user-drawn strokes for consistency.
- Custom CNN Model: Lightweight neural network trained on stroke data.
- Accuracy Scoring: Provides % match with correct letter.
- Recognition Result: Identifies if drawn letter matches the template.
- Fast & Offline: Runs directly on device for instant evaluation.
- Scalable: Can extend to other Indian scripts beyond Malayalam.

MODULES

```
onPressed: () {
  Navigator.push(
    context,
    MaterialPageRoute(
      builder: (_) => VerificationScreen(
        expectedLetter: widget.letter.character,
        modelPath: 'assets/models/ഗ്class.tflite',
      ), // VerificationScreen
    ), // MaterialPageRoute
  );
},
```

MODULES

```
double confidence = resultMap["confidence"];

String predictedLetter = classLabels[predictedIndex];
print("Predicted Letter ::::: ${predictedLetter}");

setState(() {
  if (predictedLetter == widget.expectedLetter) {
    if (confidence > 0.75) {
      result = "Excellent! 🎉";
    } else {
      result = "Almost perfect! 👍";
    }
  } else {
    if (confidence > 0.6) {
      result = "That looks close. Try adjusting slightly.";
    } else {
      result = "Let's try that again.";
    }
  }
});
}
```

MODULES

```
Epoch 21/40
60/60 ━━━━━━━━━━━━━━━━━━━━ 9s 156ms/step - accuracy: 0.9455 - loss: 0.2128 - val_accuracy: 0.5042 - val_loss: 1.4855 - learning_rate: 7.5000e-05
Epoch 22/40
60/60 ━━━━━━━━━━━━━━━━━━━━ 9s 157ms/step - accuracy: 0.9300 - loss: 0.2599 - val_accuracy: 0.5917 - val_loss: 1.0598 - learning_rate: 7.5000e-05
Epoch 23/40
60/60 ━━━━━━━━━━━━━━━━━━━━ 9s 144ms/step - accuracy: 0.9374 - loss: 0.2328 - val_accuracy: 0.9083 - val_loss: 0.2462 - learning_rate: 3.7500e-05
```

```
Start coding or generate with AI.
```

```
val_loss, val_acc = model.evaluate(val_generator)
print(f"\nValidation Accuracy: {val_acc*100:.2f}%")
```

```
15/15 ━━━━━━━━━━━━━━━━━━━━ 1s 91ms/step - accuracy: 0.9243 - loss: 0.2194

Validation Accuracy: 92.92%
```

MODULES

4. Feedback & Progress Tracking Module

- Instant Feedback: Shows success/failure message after each attempt.
- History Tracking: Logs practice attempts, accuracy, and time spent.
- Progress Dashboard: Graphs, streaks, and completion levels.
- Motivational Rewards: Badges, milestones, and practice streaks.
- Database Integration: Stores progress in Firestore/Firebase.

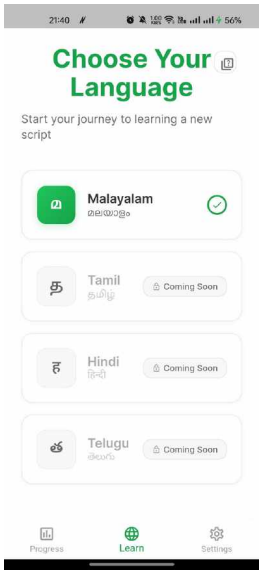
MODULES

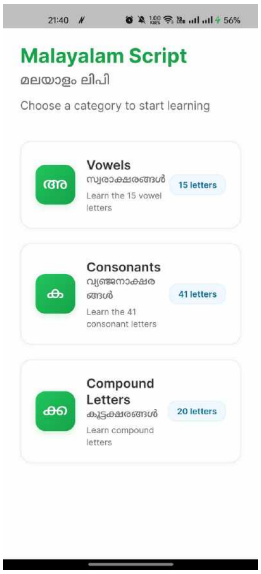
```
void _updateStreak(DateTime currentISTDate) {
  // Normalize IST dates to compare only the calendar day (year, month, day)
  final todayIST = DateTime(currentISTDate.year, currentISTDate.month, currentISTDate.day);

  if (_lastPracticeDate == null) {
    // First time user - start with streak of 1
    _dayStreak = 1;
    return;
  }
```

MODULES

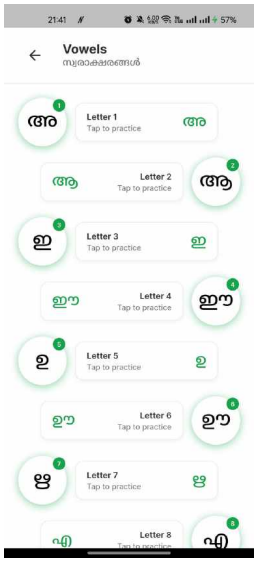
```
Future<void> _saveProgress() async {
  // Save unique completed letters as comma-separated string
  await _prefs.setCompletedLettersString(_completedLettersSet.join(','));
  await _prefs.setCompletedLetters(_completedLettersSet.length); // Also save count for compatib
  await _prefs.setDayStreak(_dayStreak);
  await _prefs.setMinutesPracticed(_minutesPracticed);
  await _prefs.setTotalAttempts(_totalAttempts);
  await _prefs.setLetterAttempts(_letterAttempts);

  if (_lastPracticeDate != null) {
    // Store in UTC format (already stored as UTC internally)
    await _prefs.setLastPracticeDate(_lastPracticeDate!.toUtc().toIso8601String());
  }

  if (_lastMinutesTrackingDate != null) {
    // Store last minutes tracking date in UTC format
    await _prefs.setLastMinutesTrackingDate(_lastMinutesTrackingDate!.toUtc().toIso8601String())
  }
}
```

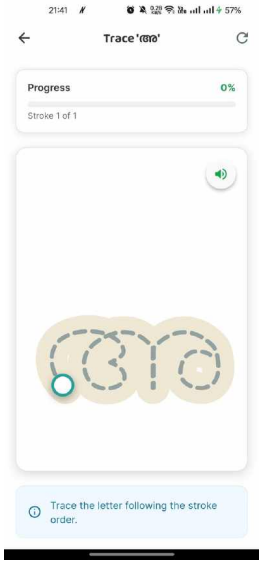
RESULTS

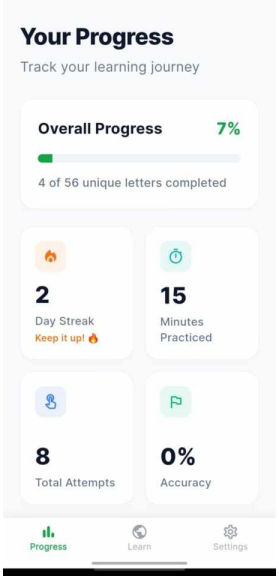




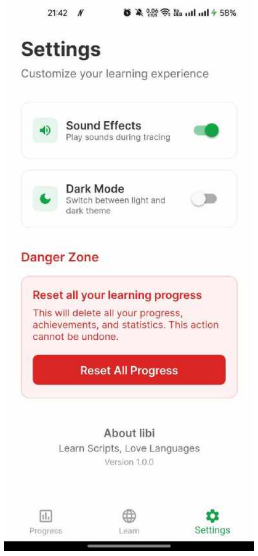
RESULTS

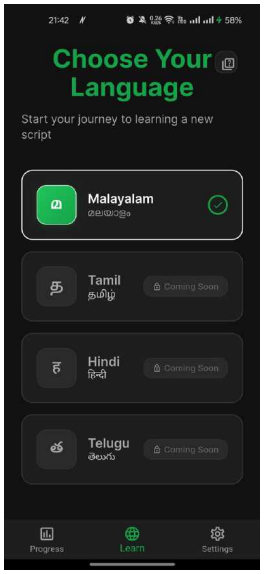




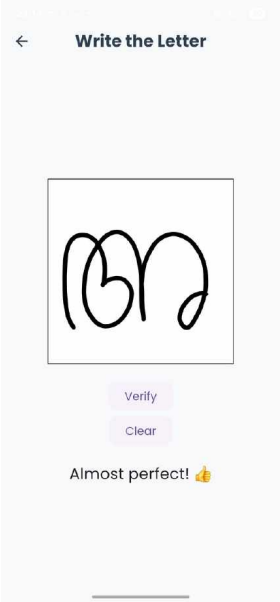


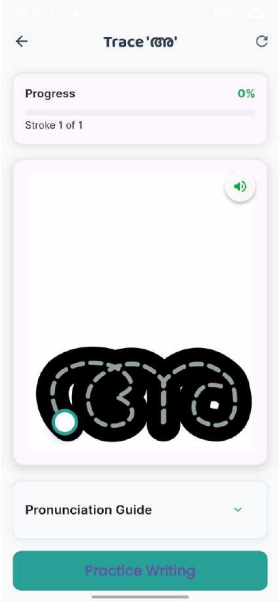
RESULTS





RESULTS





OUTPUT

- Interactive Script Learning App
  - A fully functional mobile app (built using Flutter) that teaches users to write Malayalam letters interactively.
- Touch-Based Tracing Interface
  - Users trace letters directly on the screen using finger or stylus to practice handwriting.
- Real-Time Feedback System
  - The app evaluates user input using the trained CNN model and displays correction hints instantly.

OUTPUT

- Audio Pronunciation Support
  - Each alphabet accompanied by correct pronunciation audio to aid learning and memorization.
- Progress Tracking Dashboard
  - A user dashboard showing completed lessons, accuracy scores, and learning milestones.
- Machine Learning Evaluation Results
  - CNN model validates user-drawn letters with expected classification accuracy and recognition results.

ASSUMPTIONS

- Users have access to an Android smartphone.
- Users have basic knowledge of using a smartphone.
- Users are willing to practice letters multiple times to improve accuracy.
- The screen size is sufficient for comfortable tracing.
- Users will hold the device in portrait mode while practicing.
- Users will use the app regularly for gradual improvement.

WORK BREAKDOWN

Abhinav Nair A (U2203009)

- Dataset collection and preprocessing
- CNN model design and training
- TFLite model conversion
- Flutter UI design

Anantharam S (U2203040)

- Dataset collection and preprocessing
- CNN model development
- Model integration with Flutter
- App testing and debugging

WORK BREAKDOWN

Athul K (U2203060)

- Dataset collection and preprocessing
- Front End Development
- SVG path creation
- TFLite Conversion

Bevan Sony Joseph (U2203067)

- Dataset collection and preprocessing
- CNN model development
- Model integration with Flutter
- SVG path creation

HARDWARE REQUIREMENTS

Development System

- Processor: Intel i5 / Ryzen 5 or higher
- RAM: Minimum 8 GB (16 GB recommended)
- Storage: 256 GB SSD minimum
- GPU (Optional): NVIDIA GPU (for faster CNN training)

Testing Systems

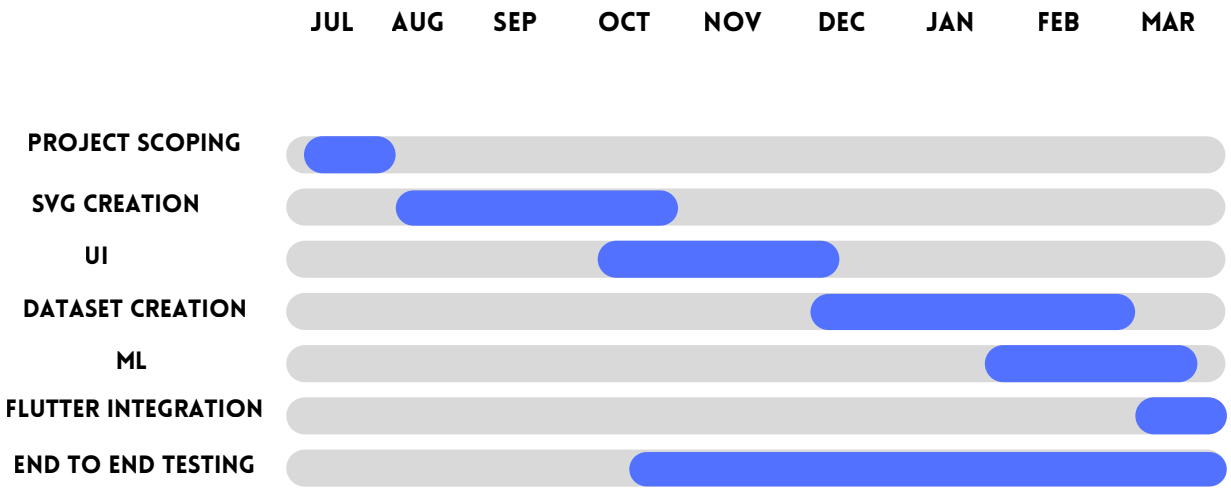
- Android smartphone
- Minimum 4 GB RAM
- Android version 8.0 or above

SOFTWARE REQUIREMENTS

Development Tools

- Flutter SDK
- Android Studio
- Visual Studio Code
- Python 3.x
- TensorFlow / Keras
- Google Colab (for training)

GANTT CHART



RISK AND ASSUMPTIONS

- Users drawing outside the canvas boundary
- Users scribbling or drawing incomplete characters
- Very small or oversized drawings
- Lack of data available publicly
- App lag on low RAM devices
- Increased model size when adding more characters
- Confusion between similar Malayalam characters

CONCLUSION

- A mobile application that helps users learn to write Indian script alphabets like Malayalam through interactive tracing. The app offers slider-based stroke animations, touch tracing practice, pronunciation support, and real-time feedback.
- It provides an intuitive, engaging, and gamified learning experience suitable for kids and beginners alike.

REFERENCE

- [1] V. K. Chauhan, S. Singh, and A. Sharma, “HCR-Net: a deep learning based script independent handwritten character recognition network,” Multimedia Tools and Applications, vol. 83, no. 32, pp. 78433–78467, Feb. 2024, doi: https://doi.org/10.1007/s11042-024-18655-5.
- [2] K. Sinha and R. Gupta, “Enhancing Handwritten Devanagari CharacterRecognition via GAN-Generated Synthetic Data,” Proceedings of the 2024 Sixteenth International Conference on Contemporary Computing, pp. 484–489, Aug. 2024, doi: https://doi.org/10.1145/3675888.3676097.
- [3] S. Singh, A. Sharma, and V. K. Chauhan, “Indic script family and its offline handwriting recognition for characters/digits and words: a comprehensive survey,” Artificial Intelligence Review, vol. 56, no. S3, pp. 3003–3055, Sep. 2023, doi: https://doi.org/10.1007/s10462-023-10597-y.

REFERENCE

- [4] A. D. R. N., P. Chetti, P. C. S. Reddy, S. Lakhanpal, A. Joshi, and S. O. V, “Nearest quad-tree (NQT) classifier: a novel framework for handwritten character recognition,” Cogent Engineering, vol. 11, no. 1, May 2024, doi: https://doi.org/10.1080/23311916.2024.2347363.

Appendix B: Vision, Mission, Programme Outcomes and Course Outcomes

Vision, Mission, Programme Outcomes and Course Outcomes

Institute Vision
To evolve into a premier technological institution, moulding eminent professionals with creative minds, innovative ideas and sound practical skill, and to shape a future where technology works for the enrichment of mankind.

Institute Mission
To impart state-of-the-art knowledge to individuals in various technological disciplines and to inculcate in them a high degree of social consciousness and human values, thereby enabling them to face the challenges of life with courage and conviction.

Department Vision
To become a centre of excellence in Computer Science and Engineering, moulding professionals catering to the research and professional needs of national and international organizations.

Department Mission
To inspire and nurture students, with up-to-date knowledge in Computer Science and Engineering, ethics, team spirit, leadership abilities, innovation and creativity to come out with solutions meeting societal needs.

Programme Outcomes (PO)
Engineering Graduates will be able to:

1. Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.

2. Problem analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development.

3. Design/development of solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required.

4. Conduct investigations of complex problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions.

5. Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems.

6. The engineer and the world: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment.

7. Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws.

8. Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.

9. Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences.

10. Project management and finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

11. Life-long learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change.

Programme Specific Outcomes (PSO)

A graduate of the Computer Science and Engineering Program will demonstrate:

PSO1: Computer Science Specific Skills

The ability to identify, analyze and design solutions for complex engineering problems in multidisciplinary areas by understanding the core principles and concepts of computer science and thereby engage in national grand challenges.

PSO2: Programming and Software Development Skills
The ability to acquire programming efficiency by designing algorithms and applying standard practices in software project development to deliver quality software products meeting the demands of the industry.

PSO3: Professional Skills
The ability to apply the fundamentals of computer science in competitive research and to develop innovative products to meet the societal needs thereby evolving as an eminent researcher and entrepreneur.

Course Outcomes (CO)

Course Outcome 1: Model and solve real world problems by applying knowledge across domains (Cognitive knowledge level: Apply).

Course Outcome 2: Develop products, processes or technologies for sustainable and socially relevant applications (Cognitive knowledge level: Apply).

Course Outcome 3: Function effectively as an individual and as a leader in diverse teams and to comprehend and execute designated tasks (Cognitive knowledge level: Apply).

Course Outcome 4: Plan and execute tasks utilizing available resources within timelines, following ethical and professional norms (Cognitive knowledge level: Apply).

Course Outcome 5: Identify technology/research gaps and propose innovative/creative solutions (Cognitive knowledge level: Analyze).

Course Outcome 6: Organize and communicate technical and scientific findings effectively in written and oral forms (Cognitive knowledge level: Apply).

Appendix C: CO-PO-PSO Mapping

COURSE OUTCOMES:

SNO	DESCRIPTION	BLOOM'S TAXONOMY LEVEL
1	CO1: Model and solve real world problems by applying knowledge across domains.	Apply
2	Develop products, processes or technologies for sustainable and socially relevant application.	Apply
3	Function effectively as an individual and as a leader in diverse teams and to comprehend and execute designated tasks.	Apply
4	Plan and execute tasks utilizing available resources within timelines, following ethical and professional norms.	Apply
5	Identify technology/research gaps and propose innovative/creative solutions.	Analyze
6	Organize and communicate technical and scientific findings effectively in written and oral forms.	Apply

MAPPING COURSE OUTCOMES (COs) – PROGRAM OUTCOMES (POs) AND PROGRAM SPECIFIC OUTCOMES (PSOs)

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2	PSO3
CO1	2	2	2	1		2	1	1	1	1	1	2	2	2
CO2	2	2	2		2	3	2	1	1	1	1	2	2	
CO3								3	2	2	1			3
CO4		1	2		3	2	3	2	2	3	1		2	2
CO5	2	3	3	1	2					1	3	3		
CO6			2			2	2	2	3	1	1			3

JUSTIFICATIONS FOR CO-PO MAPPING

CO-PO Mapping	Mapping (Low/Medium/High)	Justification
101003/ CS822U.CO1-PO1	Medium	Applies engineering knowledge, mathematics, and computing fundamentals to model and solve real-world problems.

CO-PO Mapping	Mapping (Low/Medium/High)	Justification
101003/ CS822U.CO1-PO2	Medium	Requires identification, formulation and analysis of engineering problems before developing solutions.
101003/ CS822U.CO1-PO3	Medium	Involves designing systems or solutions to address identified engineering needs.
101003/ CS822U.CO1-PO4	Low	Limited investigation or experimentation may be required to validate the developed models or solutions.
101003/ CS822U.CO1-PO6	Medium	Solutions developed may influence societal, environmental, or sustainability aspects of engineering applications.
101003/ CS822U.CO1-PO7	Medium	Encourages consideration of ethical and professional responsibilities while developing solutions.
101003/ CS822U.CO1-PO8	Low	Some level of teamwork may be required while working on modeling or project tasks.
101003/ CS822U.CO1-PO9	Low	Requires communication of results and findings through reports or presentations.
101003/ CS822U.CO1-PO10	Low	Involves basic planning and execution of tasks while implementing solutions.
101003/ CS822U.CO1-PO11	Low	Encourages independent learning and adapting new knowledge to solve real-world problems.
101003/ CS822U.CO1-PSO1	Medium	Uses core computer science principles to analyze problems and design appropriate technical solutions.
101003/ CS822U.CO1-PSO2	Medium	Involves algorithm design and application of software development practices to implement solutions.
101003/ CS822U.CO1-PSO3	Medium	Applies computer science fundamentals in research or innovative problem solving for societal needs.
101003/ CS822U.CO2-PO1	Medium	Utilizes engineering knowledge to design and develop products or systems.
101003/ CS822U.CO2-PO2	Medium	Requires analysis of user needs and technological feasibility before development.
101003/ CS822U.CO2-PO3	Medium	Focuses on design and development of solutions addressing real-world requirements.
101003/ CS822U.CO2-PO5	Medium	Requires use of modern engineering and IT tools for system or product development.
101003/ CS822U.CO2-PO6	High	Emphasizes sustainability and societal impact during solution development.
101003/ CS822U.CO2-PO7	Medium	Ethical and professional responsibilities are considered when developing socially relevant technologies.

CO-PO Mapping	Mapping (Low/Medium/High)	Justification
101003/ CS822U.CO2-PO8	Low	Development activities may require collaboration among team members.
101003/ CS822U.CO2-PO9	Low	Involves communicating design ideas and development outcomes.
101003/ CS822U.CO2-PO10	Low	Requires basic planning and coordination during development tasks.
101003/ CS822U.CO2-PO11	Low	Encourages continuous learning of emerging technologies while building solutions.
101003/ CS822U.CO2-PSO1	Medium	Applies computer science fundamentals to design technically feasible solutions.
101003/ CS822U.CO2-PSO2	Medium	Uses programming and software development practices to implement solutions.
101003/ CS822U.CO3-PO8	High	Strongly relates to functioning effectively as a member or leader in multidisciplinary teams.
101003/ CS822U.CO3-PO9	Medium	Requires effective communication among team members during collaboration.
101003/ CS822U.CO3-PO10	Medium	Team leadership involves project coordination and task management.
101003/ CS822U.CO3-PO11	Low	Team interaction supports learning from peers and adapting new approaches.
101003/ CS822U.CO3-PSO3	High	Professional collaboration and teamwork support innovative computer science solutions.
101003/ CS822U.CO4-PO2	Low	Requires analysis and planning before executing project tasks.
101003/ CS822U.CO4-PO3	Medium	Involves designing and implementing project solutions based on planned activities.
101003/ CS822U.CO4-PO5	High	Requires effective use of modern tools and resources during project execution.
101003/ CS822U.CO4-PO6	Medium	Considers societal and environmental impact while implementing solutions.
101003/ CS822U.CO4-PO7	High	Emphasizes adherence to ethical practices and professional responsibilities.
101003/ CS822U.CO4-PO8	Medium	Task execution may involve teamwork and coordination among team members.
101003/ CS822U.CO4-PO9	Medium	Requires communication of progress, outcomes and documentation.
101003/ CS822U.CO4-PO10	High	Focuses on project management including scheduling, resource allocation and task execution.

CO-PO Mapping	Mapping (Low/Medium/High)	Justification
101003/ CS822U.CO4-PO11	Low	Encourages adaptability and continuous learning during project implementation.
101003/ CS822U.CO4-PSO2	Medium	Uses software development practices to execute and manage project tasks.
101003/ CS822U.CO4-PSO3	Medium	Applies computer science knowledge in practical project development environments.
101003/ CS822U.CO5-PO1	Medium	Uses engineering knowledge to understand technological limitations and opportunities.
101003/ CS822U.CO5-PO2	High	Requires deep problem analysis and identification of research gaps.
101003/ CS822U.CO5-PO3	High	Proposes innovative design solutions to address identified gaps.
101003/ CS822U.CO5-PO4	Low	May involve limited investigation or validation of proposed ideas.
101003/ CS822U.CO5-PO5	Medium	Uses appropriate tools and techniques to analyze and develop innovative solutions.
101003/ CS822U.CO5-PO10	Low	Requires minimal planning while proposing new solution approaches.
101003/ CS822U.CO5-PO11	High	Strongly promotes independent learning and exploration of emerging technologies.
101003/ CS822U.CO5-PSO1	High	Applies computer science fundamentals to identify and solve complex technical problems.
101003/ CS822U.CO6-PO3	Medium	Requires presentation of design concepts and developed solutions.
101003/ CS822U.CO6-PO6	Medium	Communication may include discussion of societal or environmental implications.
101003/ CS822U.CO6-PO7	Medium	Requires responsible and ethical communication of technical results.
101003/ CS822U.CO6-PO8	Medium	Communication supports collaboration within team environments.
101003/ CS822U.CO6-PO9	High	Strongly relates to effective written and oral communication of technical findings.
101003/ CS822U.CO6-PO10	Low	Requires structured reporting and documentation in project management contexts.
101003/ CS822U.CO6-PO11	Low	Encourages continuous improvement in communication and presentation skills.
101003/ CS822U.CO6-PSO3	High	Communication of innovative computer science solutions and research outcomes.